

Text Splitters in LangChain

1. Introduction

After loading documents (PDFs, text, websites), we face a big problem:

👉 Documents are **too large** for LLMs to handle directly.

🚨 Why this is a problem:

- LLMs have **token limits**
 - Large text = slower + expensive
 - Hard to retrieve relevant information
-

👉 Solution: **Text Splitters**

2. Text Splitting (Core Idea)

✅ What is Text Splitting?

Text splitting means:

Breaking large text into **smaller chunks**

💡 Simple Example:

Original text:

This is a very long document containing multiple topics...

After splitting:

Chunk 1 → "This is a very long..."

Chunk 2 → "document containing..."

Chunk 3 → "multiple topics..."

🧠 Why we split text:

1. Fit within LLM token limits
 2. Improve retrieval accuracy
 3. Reduce cost and latency
 4. Better embeddings
-

👉 Text splitting is a **critical step in RAG**

🔄 Where It Fits in RAG

Document → Loader → Text Splitter → Chunks → Embeddings → Vector DB

👉 Without splitting → system performs poorly ❌

👉 With splitting → precise answers ✅

3. Length-Based Text Splitting

✅ **What it is:**

Split text based on **fixed size (length)**

Example:

- Each chunk = 500 characters
 - Or 1000 tokens
-

Code Example:

```
from langchain.text_splitter import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
```

💡 **Important Concept: Chunk Overlap**

👉 Overlap means repeating some part in next chunk

Example:

Chunk 1:

AI is transforming industries...

Chunk 2:

transforming industries by improving...

✅ **Why overlap?**

- Keeps context connected
 - Prevents loss of meaning
-

✅ **Pros:**

- Simple
 - Fast
-

✗ Cons:

- Can break sentences
 - Context may be lost
-

4. Text-Structure Based Splitting

✓ What it is:

Split text based on **natural structure**

Structure includes:

- Paragraphs
 - Sentences
 - Line breaks
-

Example:

Instead of random chunks:

Split by paragraphs

💡 Why better?

- Maintains readability
 - Preserves meaning
-

Code Idea:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

🧠 How it works:

- Try splitting by paragraph
 - If too big → split by sentence
 - If still big → split by words
-

✓ Pros:

- Smarter than length-based
 - Keeps logical flow
-

✗ Cons:

- Slightly more complex
-

5. Document-Structure Based Splitting

✅ What it is:

Split based on **document format**

Examples:

For PDFs:

- Split by pages

For Markdown:

- Split by headings (#, ##)

For HTML:

- Split by tags
-

💡 Example:

Markdown:

Introduction

Text...

Methods

Text...

Splits into:

- Introduction section
 - Methods section
-

✅ Why useful?

- Keeps semantic sections intact
 - Better for structured documents
-

✅ Use Case:

- Books
 - Reports
 - Documentation
-

6. Semantic Meaning Based Splitting

✅ What it is:

Split text based on **meaning**, not just size or structure

💡 Idea:

Group sentences that are **related in meaning**

Example:

Text:

AI is growing fast. Machine learning is a subset of AI. Cooking recipes are fun.

Splitting:

- Chunk 1 → AI + ML
 - Chunk 2 → Cooking
-

👉 Not random — based on meaning

 How it works:

Uses:

- Embeddings
 - Similarity comparison
-

 Pros:

- Best context understanding
 - High-quality retrieval
-

 Cons:

- Slower
 - More complex
 - Requires embeddings
-

 Comparison (Very Important)

Method	Based On	Pros	Cons
Length-Based	Size	Simple, fast	Breaks meaning
Text-Structure	Paragraphs	Better flow	Slight complexity
Document-Structure	Format	Keeps sections	Format dependent
Semantic-Based	Meaning	Best quality	Slow, complex

 Big Picture Understanding

Without Text Splitting:

✗ Large document

✗ Poor retrieval

✗ Wrong answers

With Text Splitting:

✓ Small chunks

✓ Better embeddings

✓ Accurate retrieval

Real-World Example

Task: Chat with PDF

Step-by-step:

1. Load PDF
 2. Split into chunks
 3. Convert to embeddings
 4. Store in vector DB
 5. Retrieve relevant chunk
 6. Send to LLM
-

👉 If chunks are bad → answers are bad

👉 If chunks are good → answers are accurate

Important Points to Remember

1. Text splitting is **essential in RAG**
 2. Chunk size affects performance
 3. Overlap helps maintain context
 4. Choose method based on data type
 5. Semantic splitting = best but expensive
-

✓ Final Summary

- **Text Splitters break large text into smaller chunks**
 - Required because LLMs have token limits
 - Types:
 1. Length-based → simple
 2. Text-structure → smarter
 3. Document-structure → format-aware
-

- 4. Semantic → meaning-based
 - Good splitting = better AI performance
-